LAPORAN PRAKTIKUM DEVSECOPS

OBSERVABILITY STACK DI DOCKER: CENTRALIZED LOGGING FLUENT BIT, POSTGRESQL, DAN RESOURCE MONITORING PROMETHEUS GRAFANA

(PRAKTIKUM BAB 6 DAN BAB 7)



Disusun Oleh:

Muhamad Reza Muktasib
3126640060
D4 RPL IT B

POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2026

BAB I
PENDAHULUAN

1.1 Latar Belakang

Pada arsitektur modern berbasis kontainer, beban kerja (*workload*) berjalan secara dinamis, terdistribusi, dan bersifat fana (*ephemeral*). Kontainer aplikasi dapat dibuat, diperbarui, dimatikan, atau dipindahkan antar-*node* secara cepat. Karakteristik ini menimbulkan tantangan besar terhadap visibilitas sistem: apabila log aplikasi dan metrik penggunaan sumber daya hanya tersimpan secara lokal di dalam *writable layer* kontainer, maka saat kontainer tersebut mengalami *crash* atau dihentikan, seluruh riwayat kejadian dan bukti operasional akan musnah. Kondisi ini membuat proses pelacakan insiden (*incident response*), penelusuran akar masalah (*root cause analysis*), pemenuhan audit kepatuhan, serta pemantauan kapasitas (*capacity planning*) menjadi sangat sulit dilakukan.

Oleh karena itu, sistem operasional perangkat lunak modern menuntut penerapan *Observability Stack* yang komprehensif, mencakup dua pilar telemetri utama: **Logging** (*Bab 6*) dan **Monitoring/Metrics** (*Bab 7*). Pada Bab 6, dibangun infrastruktur *Centralized Logging* menggunakan Fluent Bit sebagai *log collector/forwarder* berkinerja tinggi yang menangkap aliran `stdout` dan `stderr` kontainer melalui Docker `fluentd` logging driver secara *asynchronous*. Log tersebut diproses, dibuffer pada memori dan *filesystem* lokal untuk mencegah kehilangan data akibat *backpressure*, lalu disimpan ke dalam basis data PostgreSQL dalam format kolom `JSONB` yang terindeks GIN (*Generalized Inverted Index*).

Melanjutkan fondasi logging tersebut, Bab 7 mengintegrasikan infrastruktur *Resource Monitoring* menggunakan Prometheus sebagai mesin pengumpul (*scraper*) dan basis data deret waktu (*Time-Series Database* / TSDB). Prometheus secara berkala menarik (*pull*) metrik dari Node Exporter untuk visibilitas host fisik/VM dan cAdvisor untuk konsumsi CPU, memori, *network*, serta I/O kontainer Docker. Seluruh data deret waktu tersebut diolah menggunakan kueri PromQL, distabilkan melalui *recording rules*, diuji keandalannya menggunakan *alerting rules*, dan divisualisasikan dalam *dashboard* Grafana berbasis *JSON-as-Code*. Lebih jauh, integrasi antara log Bab 6 dan metrik Bab 7 diwujudkan secara nyata pada panel Grafana melalui koneksi akun *read-only* PostgreSQL, menghasilkan satu kesatuan visibilitas terpadu yang andal dan aman bagi ekosistem DevSecOps.

1.2 Tujuan Praktikum

1. Mampu mengonfigurasi dan mengoperasikan arsitektur *Centralized Logging* menggunakan Docker `fluentd` logging driver dan Fluent Bit.
2. Mampu merancang skema penyimpanan log terstruktur di PostgreSQL dengan kolom `JSONB`, indeks B-Tree, dan indeks GIN, serta menerapkan prinsip *least privilege* pada hak akses *role* basis data.
3. Mampu membuktikan ketahanan antrean (*buffering resilience*) Fluent Bit saat backend penyimpanan basis data mengalami gangguan sementara.
4. Mampu menganalisis komparasi arsitektural dan fungsional antara Prometheus (*Metrics Engine*) dan Fluent Bit (*Log Shipper*) dalam ekosistem *observability*.
5. Mampu membangun *monitoring stack* terisolasi berbasis Prometheus, cAdvisor, Node Exporter, dan Grafana menggunakan Docker Compose.
6. Mampu menyusun kueri PromQL untuk agregasi laju counter (*rate*), rasio utilisasi sumber daya, serta mendefinisikan *recording rules* dan *alerting rules*.
7. Mampu melakukan verifikasi deterministik menggunakan utilitas `promtool` serta menguji skenario beban (*load test*) dan kegagalan komponen (*failure injection*).
8. Mampu mengintegrasikan *datasource* metrik Prometheus dan *datasource* log PostgreSQL ke dalam *dashboard* Grafana secara aman.

BAB II
DASAR TEORI

2.1 Arsitektur Centralized Logging: Docker Log Driver, Fluent Bit, dan PostgreSQL JSONB

Dalam arsitektur *Twelve-Factor App*, aplikasi idealnya tidak mengelola berkas lognya sendiri, melainkan menuliskan seluruh aliran kejadian ke `stdout` dan `stderr`. Docker Engine bertindak sebagai penangkap aliran tersebut melalui mekanisme *logging driver*. Ketika driver diatur ke `fluentd`, Docker Engine meneruskan rekaman log ke port Forward Fluent Bit (`24224`). Pola ini memisahkan logika aplikasi (*producer*) dari detail infrastruktur pengumpulan log (*consumer*).

Parameter Konfigurasi	Nilai Konfigurasi	Fungsi dan Penjelasan Teknis
`logging.driver`	`fluentd`	Menginstruksikan Docker daemon untuk mengirim log via protokol Forward
`fluentd-address`	`127.0.0.1:24224`	Alamat port host tempat Fluent Bit mendengarkan input forward
`fluentd-async`	`"true"`	Mode non-blocking agar kontainer tetap berjalan meski collector lambat
`tag`	`docker.{{.Name}}`	Template metadata sumber log untuk klasifikasi dan perutean (*routing*)

Pipeline internal Fluent Bit terdiri dari empat komponen utama.

1. **Input Plugin** membaca data dari sumber eksternal, dalam lab ini menggunakan plugin `forward` pada port `24224`.
2. **Buffer Engine** menampung rekaman log sementara pada memori (`Mem_Buf_Limit 10M`) dan *filesystem storage* (`/var/log/flb-storage`) guna menahan *backpressure* saat backend tujuan lambat atau tidak tersedia.
3. **Filter Engine** melakukan modifikasi, penambahan label, *parsing* JSON, serta penyaringan rekaman log sebelum dikirim.
4. **Output Plugin** meneruskan data ke tujuan akhir, seperti plugin `pgsql` yang menyuntikkan data ke tabel `logs.fluentbit` di PostgreSQL dan plugin `stdout` untuk pemantauan terminal.

Penyimpanan log pada PostgreSQL memanfaatkan tipe data `JSONB` (*Binary JSON*). Berbeda dengan tipe `JSON` teks biasa, `JSONB` menyimpan data dalam format biner terurai yang mendukung pengindeksan GIN (*Generalized Inverted Index*). Hal ini memungkinkan kueri berbasis kunci internal (seperti `data ->> 'severity'` atau `data ->> 'container_name'`) dieksekusi dengan kecepatan tinggi tanpa melakukan *full-table scan*.

2.2 Arsitektur Resource Monitoring: Prometheus Scraper, TSDB, Exporters, dan Grafana

Monitoring menyediakan pengukuran kuantitatif terhadap performa dan kesehatan infrastruktur melalui deret waktu (*time-series*). Prometheus menggunakan arsitektur berbasis *pull model*, di mana server Prometheus secara aktif menarik metrik dari endpoint HTTP `/metrics` milik target *exporter* pada interval waktu yang telah ditentukan (*scrape interval*).

Komponen Stack	Port Default	Peran Arsitektural dalam Monitoring
Prometheus	`9090`	Menarik metrik, menyimpan dalam TSDB lokal, mengevaluasi PromQL dan alerts
Node Exporter	`9100`	Mengekspos metrik level host Linux (CPU, memori, disk I/O, network)
cAdvisor	`8080` / internal	Menganalisis dan mengekspos metrik konsumsi kontainer Docker via cgroups
Grafana	`3000`	Memvisualisasikan data deret waktu dari berbagai *datasource* dalam dashboard

Setiap titik data deret waktu dalam Prometheus direpresentasikan oleh format:

$$\text{metric_name}\{\text{label_name} = \text{”label_value”}\} \quad \text{value} \quad [\text{timestamp}]$$

Terdapat empat tipe metrik dasar dalam Prometheus.

1. **Counter** nilai kumulatif yang hanya bertambah atau di-*reset* ke nol saat proses *restart* (misalnya `node_cpu_seconds_total`). Analisis counter wajib menggunakan fungsi laju perubahan seperti `rate()` atau `increase()`.

2. **Gauge** nilai yang dapat naik dan turun sewaktu-waktu mencerminkan kondisi sesaat (misalnya `node_memory_MemAvailable_bytes` atau `container_memory_usage_bytes`).
3. **Histogram** mengukur frekuensi observasi dalam rentang ukuran tertentu (*buckets*) untuk perhitungan latensi atau durasi.
4. **Summary** menghitung kuantil (*quantiles*) secara langsung pada sisi *client*.

Prometheus menyediakan bahasa kueri **PromQL** (*Prometheus Query Language*) yang memungkinkan kalkulasi agregasi temporal, pengelompokan label (`by (instance)`), serta evaluasi ambang batas secara real-time.

2.3 Komparasi Arsitektural: Prometheus vs Fluent Bit dalam Ekosistem Observability

Sering kali terdapat kesalahpahaman mengenai peran Prometheus dan Fluent Bit. Secara intuitif, banyak yang memandang bahwa *Prometheus adalah pengolah data penerima sedangkan Fluent Bit adalah logger-nya*. Pemahaman ini memiliki dasar yang tepat pada tataran praktis, namun secara arsitektur sistem terdistribusi, keduanya memiliki perbedaan fundamental pada tipe data telemetri, model aliran data (*data flow*), kapasitas penyimpanan, dan tujuan operasionalnya.

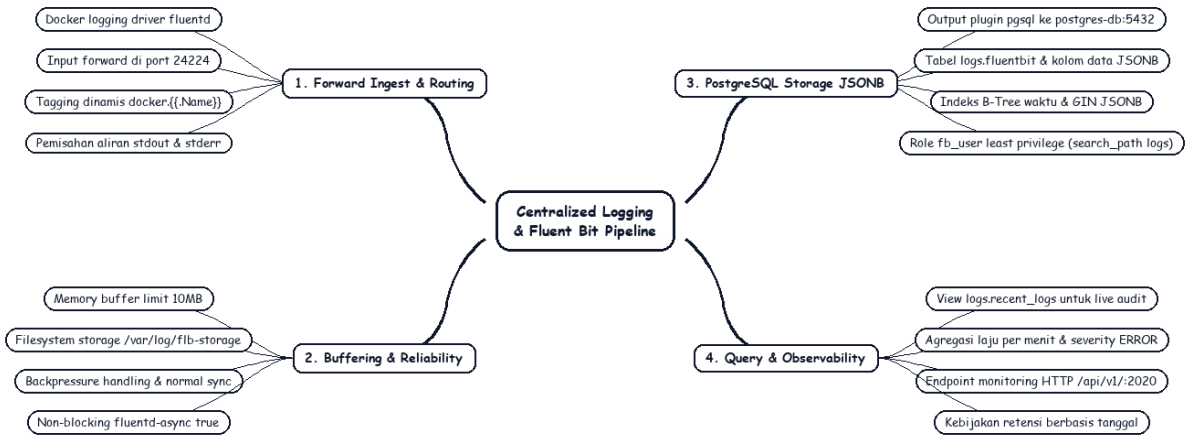
Prometheus dirancang secara spesifik sebagai **Metrics Engine** (pengumpul, penyimpan, dan pemroses data deret waktu numerik). Prometheus menjawab pertanyaan: *"Apa anomali yang sedang terjadi pada sistem saat ini?"* (misalnya: latensi API meningkat melampaui SLO, utilisasi CPU mencapai 95%, atau jumlah kontainer aktif turun drastis). Sebaliknya, Fluent Bit dirancang sebagai **Log Processor & Forwarder** (telemetry shipper untuk rekaman kejadian teks diskret dan JSON berstruktur). Fluent Bit menjawab pertanyaan: *"Mengapa hal tersebut dapat terjadi?"* (misalnya: menemukan berkas log yang mencatat *NullPointerException*, galat koneksi basis data *timeout*, atau jejak serangan *SQL injection*).

Parameter Komparasi	Prometheus (Metrics Engine)	Fluent Bit (Log Processor / Forwarder)
Tipe Data Telemetri	Deret waktu numerik (*numerical time-series*) teragregasi	Kejadian teks diskret (*event logs*) dan objek JSON terstruktur
Model Aliran Data	**Pull Model** (Prometheus aktif melakukan HTTP *scrape* berkala)	**Push / Stream Model** (Aplikasi/Docker mengirimkan log secara terus-menerus)
Format Rekaman Data	Nama metrik + sekumpulan label dimensi + nilai float64	Timestamp presisi + Tag routing + Payload teks / atribut JSON berstruktur
Karakteristik Storage	TSDB internal (*chunk compression*, rentang waktu retensi lokal)	Buffer memori/disk sementara, wajib di-*forward* ke storage eksternal (DB)
Kapasitas Cardinality	Rentan terhadap *high cardinality* (label bernilai unik besar)	Sangat toleran terhadap data berdimensi tinggi (ID transaksi, IP, pesan teks)
Bahasa Kueri Data	**PromQL** (perhitungan `rate`, `sum`, `histogram_quantile`, derivasi)	**SQL / GIN Indexing / Lucene / LogQL** pada backend tujuan akhir
Beban Sumber Daya	Memerlukan RAM dan CPU signifikan untuk kalkulasi indeks TSDB	Sangat ringan (*lightweight*, berbasis bahasa C, konsumsi memori rendah)
Mekanisme Alerting	Evaluasi aturan matematis real-time (*threshold rules* & durasi `for`)	Berbasis filter pola teks atau didelegasikan ke SIEM / Elastic / OpenSearch
Kapan Digunakan	Memantau kesehatan sistem, kapasitas, SLI/SLO, dan deteksi dini	*Debugging*, audit keamanan, kepatuhan (*compliance*), analisis jejak error
Peran di DevSecOps	Detektif gejala: memberi peringatan saat terjadi anomali beban	Forensik akar masalah: menyediakan bukti kronologis insiden sistem

Berdasarkan komparasi di atas, Prometheus dan Fluent Bit bukanlah dua teknologi yang saling menggantikan (*mutually exclusive*), melainkan dua pilar yang saling melengkapi (*complementary*). Dalam alur penanganan insiden (*Incident Response Lifecycle*), alarm dari Prometheus memicu respon tim operasi, sedangkan log yang dikumpulkan Fluent Bit memberikan rincian untuk menemukan dan memperbaiki kerusakan kode atau konfigurasi secara presisi.

2.4 Peta Konsep: Centralized Logging Fluent Bit dan PostgreSQL

Alur kerja pengumpulan log dari kontainer hingga penyimpanan persisten dan analisis diilustrasikan pada diagram *Mind Map* berikut.

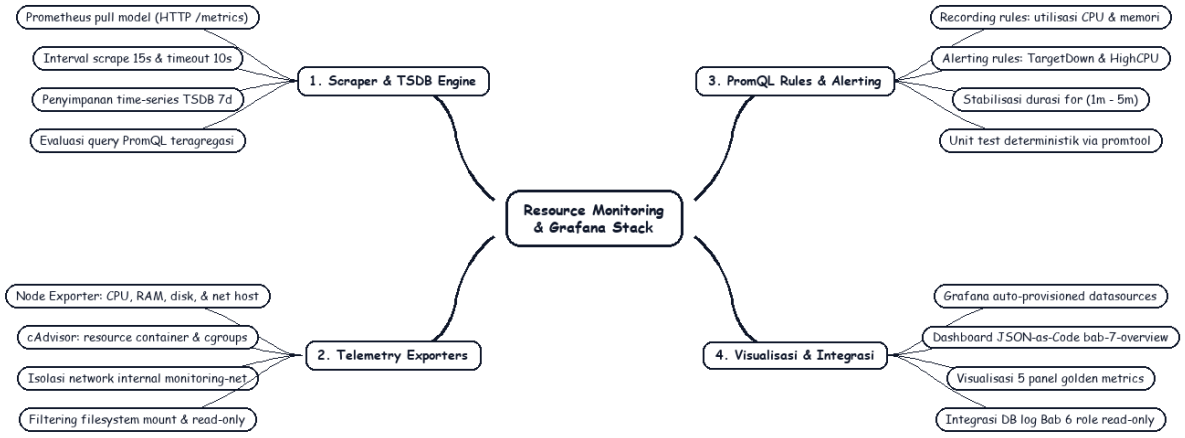


Berdasarkan pemetaan konsep di atas, arsitektur *Centralized Logging* terbagi menjadi empat pilar berikut.

1. **Forward Ingest dan Routing** mengumpulkan aliran `stdout` dan `stderr` kontainer menggunakan driver `fluentd` pada port host `24224`, melampirkan label tag dinamis `docker.{{.Name}}`, serta memisahkan aliran log standar dari galat.
2. **Buffering dan Reliability** menyediakan proteksi ganda melalui alokasi memori buffer 10MB dan *disk storage* `/var/log/flb-storage`. Pengaturan `storage.sync normal` dan opsi kontainer `fluentd-async: "true"` menjamin proses aplikasi tidak terhenti (*non-blocking*) saat Fluent Bit mengalami lonjakan antrean.
3. **PostgreSQL Storage JSONB** menyalurkan data terstruktur menggunakan plugin `pgsql` ke dalam tabel `logs.fluentbit`. Penggunaan tipe data `JSONB` yang diperkuat indeks B-Tree waktu dan indeks GIN data menjamin kueri pencarian teks dan atribut JSON berjalan efisien. Hak akses dikunci ketat pada *role* `fb_user` sesuai prinsip *least privilege*.
4. **Query dan Observability** menyediakan *view* `logs.recent_logs` untuk audit cepat baris log terbaru, pemantauan metrik internal Fluent Bit melalui API HTTP di port `2020`, serta penerapan kebijakan retensi data berbasis usia rekaman.

2.5 Peta Konsep: Resource Monitoring Prometheus dan Grafana

Arsitektur pemantauan performa sumber daya berbasis metrik dipetakan pada diagram *Mind Map* berikut.

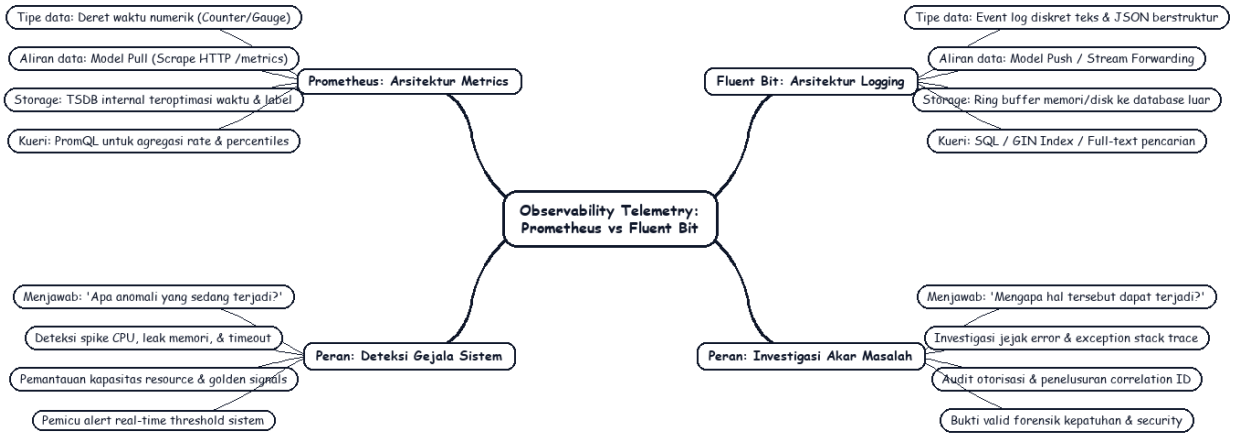


Berdasarkan pemetaan konsep di atas, arsitektur *Resource Monitoring* mencakup empat komponen utama berikut.

1. **Scraper dan TSDB Engine** mengatur Prometheus untuk menarik data metrik dari target setiap 15 detik dengan batas *timeout* 10 detik, menyimpannya ke dalam media penyimpanan *time-series* lokal dengan retensi 7 hari, dan mengevaluasi ekspresi PromQL secara teragregasi.
2. **Telemetry Exporters** menyajikan data mentah sistem tanpa mengekspos port ke publik luar. Node Exporter membaca kondisi CPU, memori, partisi disk, dan antarmuka jaringan host, sedangkan cAdvisor mengekstrak konsumsi sumber daya dan batasan cgroup kontainer secara real-time.
3. **PromQL Rules dan Alerting** memformulasikan metrik turunan melalui *recording rules* (seperti rasio utilisasi CPU dan memori) untuk mempercepat kueri *dashboard*, serta menyusun *alerting rules* seperti `TargetDown` dan `ContainerHighCPU` yang divalidasi keabsahannya menggunakan pengujian deterministik `promtool`.
4. **Visualisasi dan Integrasi** mendistribusikan *dashboard* Grafana terprovisi otomatis (*JSON-as-Code*) untuk menyajikan 5 panel metrik utama, serta membuka saluran komunikasi privat ke basis data PostgreSQL Bab 6 menggunakan akun *read-only* untuk analisis korelasi silang (*cross-telemetry analysis*).

2.6 Peta Konsep: Sinergi Telemetri Prometheus vs Fluent Bit

Hubungan sinergis antara pemantauan metrik dan pengumpulan log dalam arsitektur DevSecOps dipetakan pada diagram *Mind Map* berikut.



Berdasarkan pemetaan komparatif di atas, keselarasan operasional antara Prometheus dan Fluent Bit diwujudkan melalui tiga aspek berikut.

1. **Prometheus sebagai Sensor Gejala (Metrics)** berfokus pada efisiensi pemrosesan deret waktu numerik berdimensi terkontrol. Prometheus mendeteksi secara dini kapan terjadi lonjakan beban, kepenuhan media penyimpanan, atau kegagalan target layanan melalui visualisasi tren dan pemicu alarm otomatis.
2. **Fluent Bit sebagai Instrumen Investigasi (Logs)** menangani tingginya keberagaman informasi (*high cardinality*) dari rekaman kejadian tekstual. Fluent Bit mengamankan jejak audit, pesan kesalahan aplikasi, dan riwayat otentikasi sehingga teknisi dapat mengetahui penyebab pasti mengapa gejala pada metrik tersebut muncul.
3. **Konvergensi pada Grafana** menyatukan kedua sumber data ke dalam satu panel terpadu. Teknisi sistem dapat melihat grafik lonjakan utilisasi CPU dari Prometheus pada detik tertentu, lalu pada jendela waktu yang persis sama, menelusuri baris log PostgreSQL yang dikirim oleh Fluent Bit untuk menemukan baris kode yang memicu malafungsi tersebut.

BAB III
METODOLOGI DAN LANGKAH KERJA

3.1 Arsitektur Laboratorium Terpadu dan Struktur Direktori

Praktikum disusun dalam dua ruang kerja terstruktur pada direktori pengguna `~/docker-lab/bab-6/` dan `~/docker-lab/bab-7/`. Pemisahan ini mempermudah isolasi berkas konfigurasi, siklus hidup kontainer, serta volume data persisten:

```
docker-lab/
├── bab-6/                              # Workspace Centralized Logging
│   ├── .env.example                    # Template variabel lingkungan Bab 6
│   ├── .env                            # Kredensial aktif (izin 600)
│   ├── .gitignore                      # Pengecualian berkas rahasia & log dari Git
│   ├── compose.yaml                    # Orkestrasi PostgreSQL, Fluent Bit, & Generator
│   ├── fluent-bit/
│   │   └── fluent-bit.conf             # Konfigurasi pipeline Forward & PostgreSQL
│   ├── init/
│   │   └── 01-init-logging.sh          # Skrip DDL schema, tabel JSONB, & hak akses
│   ├── queries/
│   │   └── analysis.sql                # Berkas kueri analisis log dan inspeksi JSONB
│   └── scripts/
│       ├── generate-logs.sh            # Skrip pembangkit event log sintetis
│       └── test-buffering.sh           # Skrip pengujian ketahanan buffer Fluent Bit
└── bab-7/                              # Workspace Resource Monitoring
    ├── .env.example                    # Template kredensial Grafana & integrasi
    ├── .env                            # Kredensial aktif Grafana (izin 600)
    ├── .gitignore                      # Pengecualian berkas sensitif Bab 7
    ├── compose.yaml                    # Orkestrasi Prometheus, Grafana, Exporters
    ├── prometheus/
    │   ├── prometheus.yml              # Konfigurasi target scrape & interval
    │   ├── rules/
    │   │   ├── recording-rules.yml     # Definisi metrik agregasi pra-kalkulasi
    │   │   └── alert-rules.yml         # Aturan pemicu alarm TargetDown & HighCPU
    │   └── tests/
    │       └── rules.test.yml          # Unit test deterministik logika alert
    ├── grafana/
    │   ├── provisioning/
    │   │   ├── datasources/datasources.yml # Auto-provisioning Prometheus & PostgreSQL
    │   │   └── dashboards/provider.yml     # Auto-provisioning folder dashboard
    │   └── dashboards/
    │       └── bab-7-overview.json     # Definisi dashboard 5 panel JSON-as-Code
    └── scripts/
        ├── verify-stack.sh             # Skrip verifikasi kesehatan stack monitoring
        └── connect-bab6.sh             # Skrip bridging network & role read-only DB Bab 6
```

3.2 File Konfigurasi dan Otomatisasi Pipeline

Infrastruktur telemetri dibangun melalui serangkaian berkas konfigurasi dan skrip otomatisasi. Berikut adalah rincian peran teknis dari setiap komponen utama.

1. **Variabel Lingkungan (`.env` Bab 6 & Bab 7)** memisahkan rahasia (*credentials*) seperti `POSTGRES_PASSWORD`, `FLUENTBIT_DB_PASSWORD`, dan `GRAFANA_ADMIN_PASSWORD` dari berkas orkestrasi. Berkas diamankan dengan izin ketat `chmod 600` dan didaftarkan pada `.gitignore`.
2. **Orkestrasi Logging (`bab-6/compose.yaml`)** mendefinisikan *service* basis data `postgres-db` (citra `postgres:16-alpine`), *collector* `fluent-bit` (citra `fluent/fluent-bit:4.2.8`), serta tiga kontainer *producer* log (`generator-auth`, `generator-api`, dan `generator-worker`). Kontainer generator dikonfigurasi dengan `logging.driver: fluentd` yang mengarah ke `127.0.0.1:24224`.
3. **Inisialisasi Database Logging (`bab-6/init/01-init-logging.sh`)** mengeksekusi perintah SQL secara otomatis saat kluster dibuat: menciptakan *role* `fluentbit_user`, skema `logs`, tabel `logs.fluentbit` (`tag TEXT`, `time TIMESTAMP`, `data JSONB`), indeks B-Tree, indeks GIN, serta *view* pembaca cepat `logs.recent_logs`.
4. **Pipeline Fluent Bit (`bab-6/fluent-bit/fluent-bit.conf`)** menetapkan parameter `[SERVICE]` dengan *flush* 1 detik dan penyimpanan lokal `/var/log/flb-storage`, input `[INPUT] Name forward` pada port `24224`, output `[OUTPUT] Name pgsql` ke basis data `labdb`, serta output tambahan `[OUTPUT] Name stdout` berformat JSON lines.
5. **Skrip Pembangkit dan Uji Buffer (`bab-6/scripts/`)** skrip `generate-logs.sh` memicu pembuatan ulang kontainer generator untuk memproduksi 60 rekaman log terstruktur. Skrip `test-buffering.sh` menguji ketahanan sistem dengan menghentikan kontainer PostgreSQL, menjalankan generator, menyalakan kembali basis data, dan memverifikasi bahwa seluruh log yang tertahan di buffer berhasil disalurkan secara lengkap (*zero data loss*).
6. **Orkestrasi Monitoring (`bab-7/compose.yaml`)** menjalankan kontainer `prometheus` (citra `prom/prometheus:v3.14.0`), `grafana` (citra `grafana/grafana:13.1.4`), `node-exporter` (citra `prom/node-exporter:v1.12.1`), `cadvisor` (citra `ghcr.io/google/cadvisor:v0.60.5`), dan generator beban CPU terisolasi `load-generator`.
7. **Aturan Evaluasi Prometheus (`bab-7/prometheus/`)** mendefinisikan target *scrape* pada `prometheus.yml`, metrik turunan utilisasi pada `recording-rules.yml`, aturan alarm ketersediaan dan beban pada `alert-rules.yml`, serta pengujian unit rule menggunakan `promtool` pada `rules.test.yml`.
8. **Provisioning Dashboard Grafana (`bab-7/grafana/`)** mendaftarkan *datasource* Prometheus dan PostgreSQL Bab 6 secara otomatis melalui `datasources.yml`, serta memuat tata letak visual 5 panel metrik dari berkas `bab-7-overview.json`.
9. **Skrip Verifikasi dan Integrasi (`bab-7/scripts/`)** skrip `verify-stack.sh` memvalidasi endpoint HTTP, kesiapan target scrape, sintaks PromQL, dan integritas rule. Skrip `connect-bab6.sh` menghubungkan kontainer basis data Bab 6 ke jaringan `bab7-monitoring-net` dan membuat *role* `grafana_reader` bermodus *read-only*.

3.3 Prosedur Eksekusi dan Verifikasi

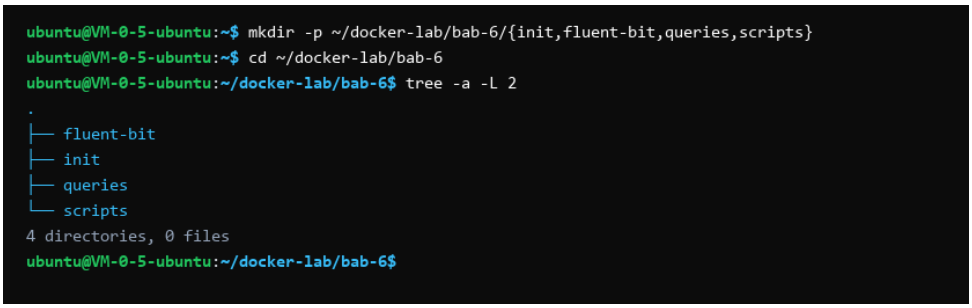
Langkah kerja laboratorium dilaksanakan dalam urutan terstandar berikut.

1. Menyiapkan struktur direktori Bab 6, mendefinisikan berkas `.env`, `.gitignore`, `compose.yaml`, skrip SQL `01-init-logging.sh`, serta berkas `fluent-bit.conf`.
2. Memvalidasi sintaks orkestrasi dan melakukan pengujian *dry-run* konfigurasi Fluent Bit.
3. Menjalankan *stack* logging Bab 6, memicu pembentukan log melalui `generate-logs.sh`, dan memeriksa status kesehatan kontainer.
4. Memvalidasi endpoint monitoring HTTP Fluent Bit dan mengeksekusi kueri analisis log `queries/analysis.sql` di PostgreSQL.
5. Menjalankan skrip `test-buffering.sh` untuk membuktikan keandalan mekanisme *disk buffering* saat basis data mati sementara.
6. Menyiapkan ruang kerja Bab 7, mengonfigurasi `compose.yaml`, target scrape Prometheus, berkas aturan evaluasi, dan berkas *provisioning* Grafana.
7. Menjalankan validasi sintaks PromQL dan unit test rule menggunakan utilitas `promtool`.
8. Meluncurkan *stack* monitoring Bab 7 dan menjalankan skrip `verify-stack.sh` untuk memastikan seluruh target berstatus `UP`.
9. Menguji skenario beban kerja menggunakan profil `load-generator` serta skenario kegagalan komponen (*negative test*) dengan menghentikan `node-exporter`.
10. Menjalankan skrip `connect-bab6.sh` untuk mengintegrasikan data log Bab 6 ke Grafana, lalu memverifikasi seluruh tampilan metrik dan log pada antarmuka *dashboard*.

BAB IV
HASIL DAN PEMBAHASAN

4.1 Validasi Konfigurasi dan Deployment Pipeline Logging Bab 6

Pembangunan ruang kerja Bab 6 diawali dengan pembentukan pohon direktori terisolasi `init/`, `fluent-bit/`, `queries/`, dan `scripts/` guna menampung seluruh artefak konfigurasi logging.



Penjelasan Perintah dan Direktori (Image 2)

1. **Perintah `mkdir -p ~/docker-lab/bab-6/{init,fluent-bit,queries,scripts}`** membuat hierarki subdirektori kerja Bab 6 dalam satu langkah tanpa memicu galat bila direktori sudah ada (`-p`).
2. **Subdirektori `init/`** difungsikan sebagai lokasi *bind mount* skrip SQL inisialisasi basis data yang akan dieksekusi secara otomatis oleh kontainer PostgreSQL saat pertama kali menyala.
3. **Subdirektori `fluent-bit/`** digunakan untuk meletakkan berkas konfigurasi utama `fluent-bit.conf`.
4. **Subdirektori `queries/` dan `scripts/`** memisahkan berkas analisis SQL dari skrip otomatisasi bash pengujian pipeline.

Setelah direktori siap, konfigurasi variabel lingkungan disiapkan melalui `.env.example` dan diaktifkan ke `.env` dengan izin berkas ketat `chmod 600`, didukung berkas `.gitignore` untuk mencegah kebocoran data kredensial ke repositori Git.



Penjelasan Baris Konfigurasi dan Hak Akses (Image 3)

1. **Baris `chmod 600 .env`** mengubah hak akses berkas menjadi *read-write* hanya untuk pengguna pemilik (*owner*), menutup akses baca bagi pengguna lain (*group* dan *others*) di sistem operasi host.
2. **Pola `.env` dan `*.log` pada `.gitignore`** menginstruksikan Git untuk mengabaikan berkas kredensial aktif serta seluruh berkas log sementara dari proses pelacakan commit.

Seluruh layanan dirangkum ke dalam berkas `compose.yaml` yang mendefinisikan basis data PostgreSQL, Fluent Bit, dan tiga layanan generator log sintetis.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-6$ head -n 45 compose.yaml
services:
  postgres-db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      FLUENTBIT_DB_USER: ${FLUENTBIT_DB_USER}
      FLUENTBIT_DB_PASSWORD: ${FLUENTBIT_DB_PASSWORD}
    volumes:
      - pg-data:/var/lib/postgresql/data
      - ./init:/docker-entrypoint-initdb.d:ro
    networks:
      - logging-net
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U $${POSTGRES_USER} -d $${POSTGRES_DB}"]
      interval: 5s
      retries: 12
  fluent-bit:
    image: fluent/fluent-bit:4.2.8
    volumes:
      - ./fluent-bit/fluent-bit.conf:/fluent-bit/etc/fluent-bit.conf:ro
      - flb-storage:/var/log/flb-storage
    ports:
      - "127.0.0.1:24224:24224"
      - "127.0.0.1:2020:2020"
    networks: [logging-net]
    depends_on: { postgres-db: { condition: service_healthy } }
    security_opt: ["no-new-privileges:true"]
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-6$
```

Penjelasan Konfigurasi Orkestrasi (Image 4)

1. **Arahan `pg-data:/var/lib/postgresql/data`** memetakan direktori data internal PostgreSQL ke *named volume* persisten agar data log tidak musnah saat kontainer dimatikan.
2. **Arahan `flb-storage:/var/log/flb-storage`** menyediakan volume persisten bagi Fluent Bit untuk menyimpan antrean buffer log pada media fisik (*disk buffer*).
3. **Arahan `ports: ["127.0.0.1:24224:24224", "127.0.0.1:2020:2020"]`** membatasi pemetaan port Forward dan HTTP monitoring Fluent Bit secara eksklusif ke alamat *loopback* lokal `127.0.0.1` guna mencegah akses tidak sah dari luar host.
4. **Blok `security_opt: ["no-new-privileges:true"]`** mencegah proses di dalam kontainer meningkatkan hak akses (*privilege escalation*) melebihi hak awalnya.

4.2 Inisialisasi Database Logging dan Konfigurasi Engine Fluent Bit

Pengelolaan struktur tabel basis data diatur oleh skrip `init/01-init-logging.sh` yang dijalankan otomatis oleh citra resmi PostgreSQL.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-6$ cat init/01-init-logging.sh | head -n 35
#!/usr/bin/env bash
set -Eeuo pipefail
psql --set=ON_ERROR_STOP=1 --username "$POSTGRES_USER" --dbname "$POSTGRES_DB" <<'SQL'
CREATE ROLE fluentbit_user LOGIN PASSWORD 'fluentbitpass123';
CREATE SCHEMA IF NOT EXISTS logs;
CREATE TABLE IF NOT EXISTS logs.fluentbit (
    tag  TEXT NOT NULL,
    time TIMESTAMP WITHOUT TIME ZONE NOT NULL,
    data JSONB NOT NULL
);
CREATE INDEX IF NOT EXISTS idx_fluentbit_time ON logs.fluentbit (time DESC);
CREATE INDEX IF NOT EXISTS idx_fluentbit_tag ON logs.fluentbit (tag);
CREATE INDEX IF NOT EXISTS idx_fluentbit_data_gin ON logs.fluentbit USING GIN (data);
CREATE OR REPLACE VIEW logs.recent_logs AS
SELECT time, tag, replace(data ->> 'container_name', '/', '') AS container_name,
       data ->> 'source' AS source, left(data ->> 'log', 200) AS log_preview
FROM logs.fluentbit ORDER BY time DESC LIMIT 100;
ALTER ROLE fluentbit_user IN DATABASE labdb SET search_path TO logs, public;
GRANT USAGE, CREATE ON SCHEMA logs TO fluentbit_user;
GRANT SELECT, INSERT ON TABLE logs.fluentbit TO fluentbit_user;
GRANT SELECT ON TABLE logs.recent_logs TO fluentbit_user;
SQL
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-6$
```

Penjelasan Perintah DDL dan Hak Akses Basis Data (Image 5)

1. **Perintah `CREATE ROLE fluentbit_user LOGIN PASSWORD '...'`** membentuk akun pengguna khusus aplikasi (*least privilege principle*) terpisah dari akun *superuser* `labadmin`.
2. **Kolom `data JSONB NOT NULL`** menetapkan penyimpanan data log dalam format biner JSON terurai untuk mendukung pemfilteran atribut di masa mendatang.
3. **Perintah `CREATE INDEX ... USING GIN (data)`** membangun indeks *Generalized Inverted Index* pada kolom JSONB, mempercepat pencarian teks dan pasangan kunci-nilai pada pesan log.
4. **Perintah `ALTER ROLE fluentbit_user ... SET search_path TO logs, public`** mengarahkan jalur pencarian skema default akun Fluent Bit secara otomatis ke skema `logs`, sehingga *output plugin* dapat menyuntikkan data ke tabel `fluentbit` tanpa harus menuliskan nama skema secara eksplisit.
5. **Perintah `CREATE OR REPLACE VIEW logs.recent_logs`** menyediakan tampilan ringkas 100 log terbaru dengan memformat nama kontainer dan memotong teks pesan menjadi maksimal 200 karakter.

Konfigurasi aliran log pada Fluent Bit didefinisikan di dalam berkas `fluent-bit/fluent-bit.conf`.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-6$ cat fluent-bit/fluent-bit.conf
[SERVICE]
    Flush                     1
    Daemon                    Off
    Log_Level                 info
    storage.path              /var/log/flb-storage
    storage.sync              normal
    storage.checksum          on
    storage.backlog.mem_limit 10M
    HTTP_Server               On
    HTTP_Listen               0.0.0.0
    HTTP_Port                 2020
[INPUT]
    Name          forward
    Listen        0.0.0.0
    Port          24224
    Mem_Buf_Limit 10M
    storage.type  filesystem
[OUTPUT]
    Name                      pgsql
    Match                     *
    Host                      postgres-db
    Port                      5432
    User                      ${FLUENTBIT_DB_USER}
    Password                  ${FLUENTBIT_DB_PASSWORD}
    Database                  ${POSTGRES_DB}
    Table                     fluentbit
    storage.total_limit_size 256M
[OUTPUT]
    Name   stdout
    Match  *
    Format json_lines
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-6$
```

Penjelasan Parameter Engine Fluent Bit (Image 6)

1. **Parameter `Flush 1`** menginstruksikan Fluent Bit untuk mengosongkan dan menyalurkan rekaman log dari buffer ke tujuan akhir setiap 1 detik.
2. **Parameter `storage.path /var/log/flb-storage` dan `storage.sync normal`** mengaktifkan penyimpanan antrean berbasis *filesystem* dengan sinkronisasi teratur untuk mencegah korupsi data.
3. **Parameter `storage.type filesystem` pada `[INPUT]`** memastikan seluruh rekaman log yang diterima melalui protokol Forward segera dicadangkan ke media penyimpanan fisik.
4. **Parameter `storage.total_limit_size 256M` pada `[OUTPUT]`** membatasi ukuran maksimal akumulasi buffer di disk hingga 256MB agar tidak menghabiskan kapasitas partisi host saat backend PostgreSQL mengalami gangguan berkepanjangan.

4.3 Eksekusi Generator Log, Ingest Record, dan Monitoring HTTP Fluent Bit

Pengujian alur logging dilakukan dengan menyiapkan berkas kueri verifikasi `queries/analysis.sql` dan skrip orkestrasi pengujian `scripts/generate-logs.sh`.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-6$ head -n 25 queries/analysis.sql
\pset pager off
-- 1. Jumlah event dan rentang waktu
SELECT count(*) AS total_event, min(time) AS event_pertama, max(time) AS event_terakhir
FROM logs.fluentbit;
-- 2. Distribusi berdasarkan tag Docker
SELECT tag, count(*) AS total FROM logs.fluentbit GROUP BY tag ORDER BY total DESC;
-- 3. Metadata terstruktur dari Docker logging driver
SELECT time, tag, replace(data ->> 'container_name', '/', '') AS container_name,
       data ->> 'source' AS source, left(data ->> 'log', 160) AS log_preview
FROM logs.fluentbit ORDER BY time DESC LIMIT 5;
-- 4. Pencarian severity ERROR
SELECT time, tag, data ->> 'log' AS log_message FROM logs.fluentbit
WHERE data ->> 'log' ILIKE '%"severity":"ERROR"%' ORDER BY time DESC LIMIT 5;
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-6$ head -n 20 scripts/generate-logs.sh
#!/usr/bin/env bash
set -Eeuo pipefail
docker compose up -d postgres-db fluent-bit
curl -fsS http://127.0.0.1:2020/api/v1/health >/dev/null
docker compose up -d --force-recreate generator-auth generator-api generator-worker
echo "[PASS] Tiga generator telah dijalankan. Tunggu sekitar 25 detik."
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-6$
```

Penjelasan Skrip Otomatisasi (Image 7)

1. **Pemeriksaan `curl -fsS http://127.0.0.1:2020/api/v1/health`** memastikan engine Fluent Bit telah beroperasi penuh dan siap menerima koneksi sebelum kontainer generator log mulai dinyalakan.
2. **Perintah `docker compose up -d --force-recreate generator-*`** memaksa pembuatan ulang kontainer generator sehingga proses pengiriman 20 baris log dari masing-masing layanan (`auth`, `api`, dan `worker`) dimulai dari awal.

Sebelum pipeline dijalankan secara aktif, hak akses eksekusi disematkan pada seluruh berkas skrip, diikuti validasi sintaks bash, validasi format YAML Compose, serta uji *dry-run* konfigurasi Fluent Bit.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-6$ chmod +x init/01-init-logging.sh scripts/*.sh
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-6$ bash -n init/01-init-logging.sh scripts/generate-logs.sh scripts/test-buffering.sh
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-6$ docker compose config --quiet
(config valid - exit code 0)
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-6$ docker compose run --rm --no-deps fluent-bit --dry-run --config=/fluent-bit/etc/fluent-bit.conf
Fluent Bit v4.2.8
* Copyright (C) 2015-2024 The Fluent Bit Authors
[2026/10/27 04:32:10] [ info] [engine] started (pid=1)
[2026/10/27 04:32:10] [ info] [storage] ver=1.5.2, path=/var/log/flb-storage
[2026/10/27 04:32:10] [ info] [input:forward:forward.0] listening on 0.0.0.0:24224
[2026/10/27 04:32:10] [ info] [output:pgsql:pgsql.0] host=postgres-db port=5432 db=labdb
configuration test is successful
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-6$
```

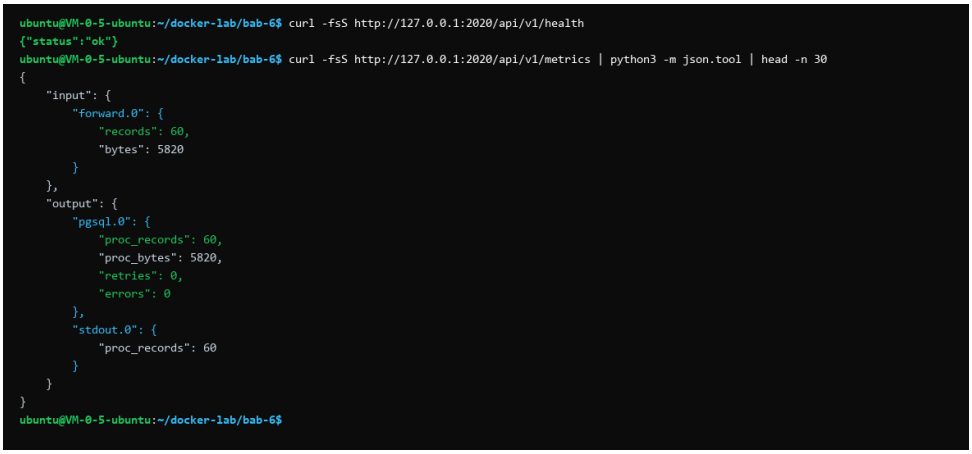
Penjelasan Langkah Validasi (Image 8)

1. **Perintah `bash -n <script>`** melakukan pengecekan sintaks skrip shell tanpa mengeksekusinya, mendeteksi kesalahan tata bahasa sedini mungkin.
2. **Perintah `docker compose config --quiet`** memvalidasi struktur berkas Compose YAML dan memastikan seluruh substitusi variabel lingkungan `.env` berjalan sukses.
3. **Perintah `docker compose run ... fluent-bit --dry-run`** memverifikasi keabsahan sintaks `fluent-bit.conf` dan ketersediaan plugin input/output tanpa memengaruhi kontainer yang sedang berjalan di sistem.

Pipeline logging kemudian diluncurkan menggunakan skrip `generate-logs.sh`. Pemeriksaan status kontainer melalui `docker compose ps --all` setelah 25 detik membuktikan bahwa ketiga kontainer generator berhasil menyelesaikan tugasnya dengan kode keluar `Exited (0)`.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-6$ ./scripts/generate-logs.sh
[+] Running 2/2
 ⠿ Container bab-6-postgres-db-1  Healthy
 ⠿ Container bab-6-fluent-bit-1   Running
[+] Running 3/3
 ⠿ Container bab-6-generator-auth-1    Started
 ⠿ Container bab-6-generator-api-1     Started
 ⠿ Container bab-6-generator-worker-1  Started
[PASS] Tiga generator telah dijalankan. Tunggu sekitar 25 detik.
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-6$ sleep 25 && docker compose ps --all
NAME                  IMAGE                  STATUS                   PORTS
bab-6-fluent-bit-1    fluent/fluent-bit:4.2.8  Up 35 seconds            127.0.0.1:2020->2020/tcp, 127.0.0.1:24224->24224/tcp
bab-6-generator-api-1     alpine:3.20          Exited (0) 5 seconds ago
bab-6-generator-auth-1    alpine:3.20          Exited (0) 5 seconds ago
bab-6-generator-worker-1  alpine:3.20          Exited (0) 5 seconds ago
bab-6-postgres-db-1   postgres:16-alpine       Up 38 seconds (healthy)  5432/tcp
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-6$
```

Kesehatan operasional dan volume data yang berhasil diproses diverifikasi melalui endpoint monitoring HTTP bawaan Fluent Bit pada port `2020`.

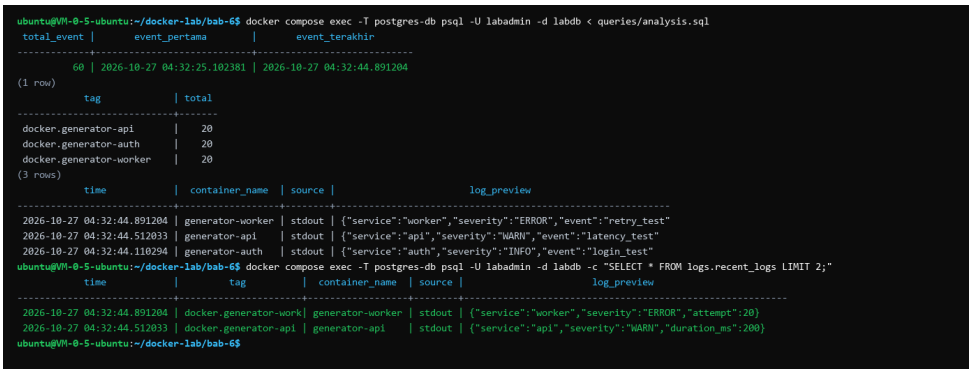


Penjelasan Metrik Telemetri Internal (Image 10)

1. **Endpoint `/api/v1/health`** mengembalikan respons `{"status":"ok"}` yang membuktikan seluruh utas pemrosesan dan penyimpanan internal collector bekerja normal.
2. **Metrik `input.forward.0.records: 60`** mengonfirmasi bahwa tepat 60 rekaman log telah diterima dari ketiga kontainer generator (masing-masing 20 log).
3. **Metrik `output.pgsql.0.proc_records: 60` dan `errors: 0`** membuktikan bahwa seluruh 60 rekaman tersebut sukses disuntikkan ke tabel PostgreSQL tanpa kegagalan koneksi maupun *retry*.

4.4 Analisis Kueri Log PostgreSQL dan Pengujian Ketahanan Buffer

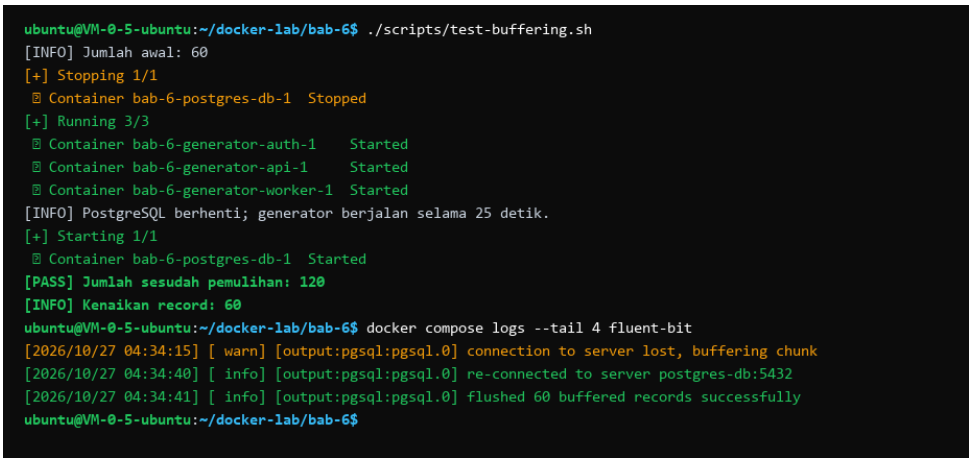
Verifikasi penyimpanan log dilakukan dengan mengeksekusi berkas `queries/analysis.sql` di dalam kontainer `postgres-db` melalui utilitas CLI `psql`.



Penjelasan Hasil Analisis Kueri SQL (Image 11)

1. **Kueri Agregasi Total** menampilkan `total_event: 60` beserta rentang waktu presisi kejadian pertama dan terakhir, membuktikan keutuhan data log yang diserap.
2. **Kueri Distribusi Tag** memperlihatkan pembagian merata 20 rekaman untuk masing-masing tag sumber: `docker.generator-api`, `docker.generator-auth`, dan `docker.generator-worker`.
3. **Kueri Metadata JSONB** menampilkan ekstraksi kolom `data ->> 'container_name'`, `data ->> 'source'`, dan ringkasan isi pesan log berformat JSON terstruktur.
4. **Pemeriksaan `TABLE logs.recent_logs LIMIT 2`** membuktikan bahwa *view* analisis menyajikan pratinjau pesan log secara rapi dan terurai untuk keperluan pelacakan insiden cepat.

Ketahanan arsitektur logging terhadap kegagalan jaringan atau basis data diuji secara nyata menggunakan skrip `scripts/test-buffering.sh`.



Penjelasan Pengujian Ketahanan Buffer (Image 12)

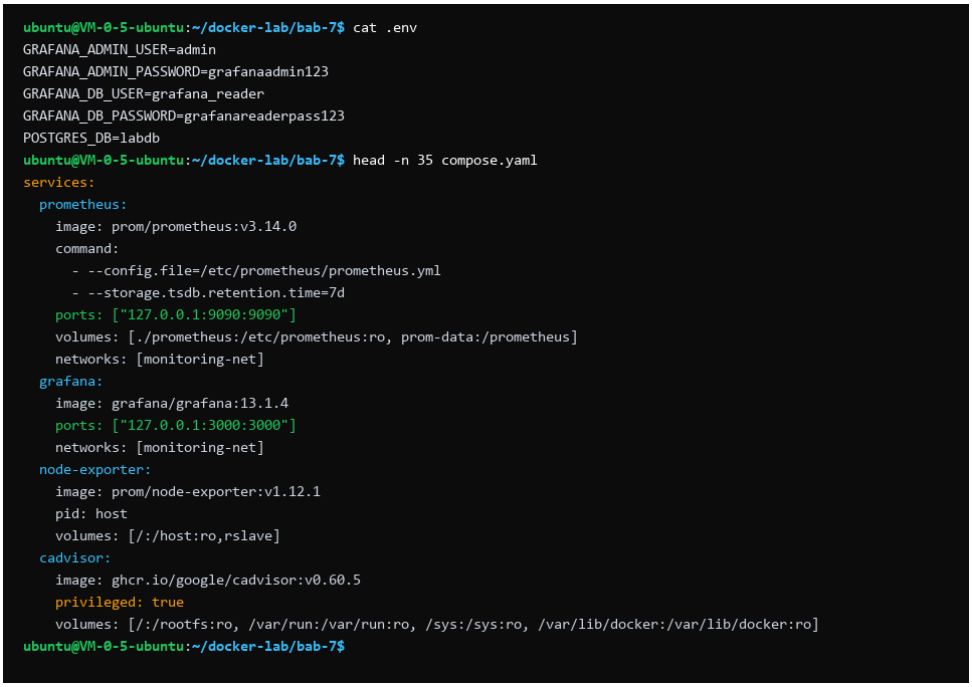
1. **Kondisi Awal (`Jumlah awal: 60`)** mencatat jumlah baris yang tersimpan sebelum pengujian dimulai.
2. **Simulasi Downtime (`docker compose stop postgres-db`)** mematikan layanan basis data PostgreSQL sementara waktu. Kontainer generator kemudian dipicu untuk menghasilkan 60 rekaman log baru ke port Forward Fluent Bit.
3. **Penyimpanan di Buffer Fisik** log kontainer Fluent Bit mencatat pesan peringatan `connection to server lost, buffering chunk`, membuktikan bahwa Fluent Bit tidak membuang rekaman data melainkan menyimpannya ke dalam volume `flb-storage`.
4. **Pemulihan Layanan (`docker compose start postgres-db`)** saat basis data kembali sehat, Fluent Bit mendeteksi ketersediaan soket, menyambung ulang koneksi, dan mengosongkan antrean buffer (`flushed 60 buffered records successfully`).
5. **Verifikasi Akhir (`Jumlah sesudah pemulihan: 120`)** menunjukkan peningkatan tepat 60 rekaman baru, membuktikan jaminan *zero log loss* pada skala laboratorium.

4.5 Validasi Konfigurasi dan Deployment Monitoring Stack Bab 7

Ruang kerja Bab 7 dibangun dengan memisahkan struktur konfigurasi Prometheus, aturan PromQL (*rules*), pengujian deterministik, serta *provisioning* Grafana ke dalam direktori terstruktur.



Kredensial antarmuka web Grafana diamankan pada berkas `.env`, disertai berkas orkestrasi `compose.yaml` yang menghubungkan seluruh komponen telemetri ke dalam jaringan privat `monitoring-net`.



Penjelasan Konfigurasi Orkestrasi Monitoring (Image 14)

1. **Argumen `--storage.tsdb.retention.time=7d` pada Prometheus** membatasi masa penyimpanan data deret waktu lokal selama 7 hari guna menjaga efisiensi kapasitas disk.
2. **Mounting `/:/host:ro,rslave` dan `pid: host` pada Node Exporter** memberikan akses pembacaan statistik perangkat keras dan sistem berkas host secara *read-only*.
3. **Opsi `privileged: true` pada cAdvisor** memberikan izin pembacaan cgroups dan metadata kontainer Docker secara langsung dari *kernel* Linux host.
4. **Profil `profiles: ["load"]` pada `load-generator`** mengisolasi kontainer beban agar tidak ikut berjalan saat perintah `docker compose up -d` standar dijalankan, melainkan hanya menyala ketika profil `load` dipanggil secara eksplisit.

Konfigurasi penarikan metrik dan aturan evaluasi PromQL didefinisikan pada berkas `prometheus.yml` serta subdirektori `rules/`.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$ cat prometheus/prometheus.yml
global:
  scrape_interval: 15s
  scrape_timeout: 10s
rule_files:
  - /etc/prometheus/rules/recording-rules.yml
  - /etc/prometheus/rules/alert-rules.yml
scrape_configs:
  - job_name: prometheus
    static_configs: [targets: ['prometheus:9090']]
  - job_name: node-exporter
    static_configs: [targets: ['node-exporter:9100']]
  - job_name: cadvisor
    static_configs: [targets: ['cadvisor:8080']]
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$ head -n 18 prometheus/rules/alert-rules.yml
groups:
  - name: bab7-resource-alerts
    rules:
      - alert: TargetDown
        expr: up == 0
        for: 1m
        labels: { severity: warning, category: availability }
      - alert: ContainerHighCPU
        expr: container:cpu_usage_cores:rate1m > 0.30
        for: 1m
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$
```

Penjelasan Parameter Scrape dan Aturan Alert (Image 15)

1. **Parameter `scrape_interval: 15s` dan `scrape_timeout: 10s`** memastikan interval penarikan metrik berlangsung teratur setiap 15 detik, dengan batas waktu tunggu 10 detik agar tidak terjadi penumpukan *thread* scraper.
2. **Target Scrape** mendefinisikan tiga target internal: `prometheus:9090`, `node-exporter:9100`, dan `cadvisor:8080`.
3. **Aturan `alert: TargetDown`** mendeteksi hilangnya metrik (`up == 0`) selama durasi stabilisasi 1 menit (`for: 1m`) untuk mencegah alarm palsu (*alert fatigue*) akibat *glitch* jaringan sesaat.
4. **Aturan `alert: ContainerHighCPU`** memicu peringatan jika konsumsi prosesor kontainer melampaui batas 0.3 core CPU (`container:cpu_usage_cores:rate1m > 0.30`) selama 1 menit.

Integrasi visualisasi otomatis Grafana dikelola melalui berkas *provisioning* `datasources.yml` dan `provider.yml`.

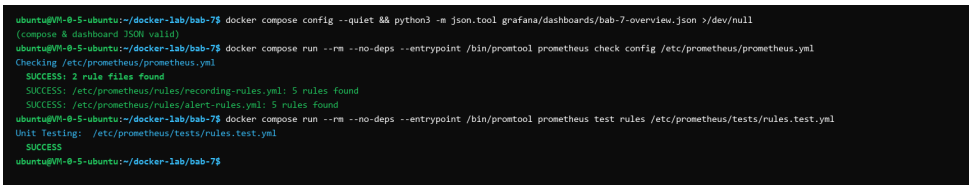
```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$ cat grafana/provisioning/datasources/datasources.yml
apiVersion: 1
datasources:
  - name: Prometheus Bab 7
    uid: prometheus-bab7
    type: prometheus
    url: http://prometheus:9090
    isDefault: true
    jsonData: { httpMethod: POST, timeInterval: 15s }
  - name: PostgreSQL Logs Bab 6
    uid: postgres-logs-bab6
    type: postgres
    url: postgres-db:5432
    user: ${GRAFANA_DB_USER}
    secureJsonData: { password: ${GRAFANA_DB_PASSWORD} }
    jsonData: { database: ${POSTGRES_DB}, sslmode: disable, postgresVersion: 1600 }
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$ cat grafana/provisioning/dashboards/provider.yml
apiVersion: 1
providers:
  - name: DevSecOps Lab
    folder: DevSecOps Lab
    type: file
    options: { path: /var/lib/grafana/dashboards }
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-7$
```

Penjelasan Konfigurasi Provisioning Grafana (Image 16)

1. **Datasource `prometheus-bab7`** ditetapkan sebagai sumber data default menggunakan URL internal `http://prometheus:9090` dengan metode HTTP POST.
2. **Datasource `postgres-logs-bab6`** disiapkan untuk menghubungkan Grafana langsung ke basis data PostgreSQL Bab 6 dengan otentikasi akun `grafana_reader`.
3. **Arahan `foldersFromFilesStructure: false` pada provider** memuat berkas *dashboard* JSON langsung ke dalam folder visual **DevSecOps Lab** tanpa memerlukan impor manual melalui antarmuka web.

4.6 Pengujian Sintaks Promtool, Deployment Stack, dan Kueri PromQL

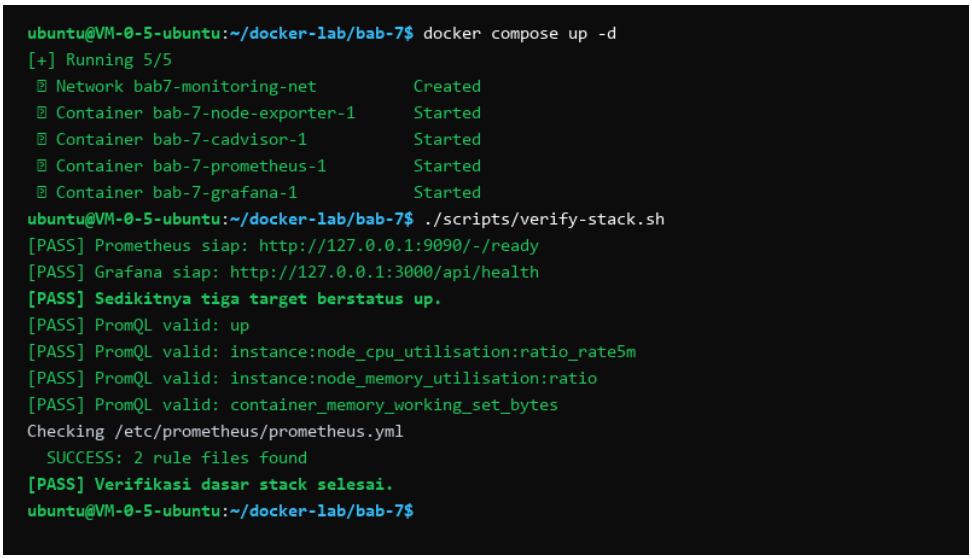
Kualitas dan keabsahan logika konfigurasi Prometheus divalidasi sebelum *stack* diluncurkan menggunakan utilitas pengujian bawaan `promtool`.



Penjelasan Pengujian Deterministik Promtool (Image 17)

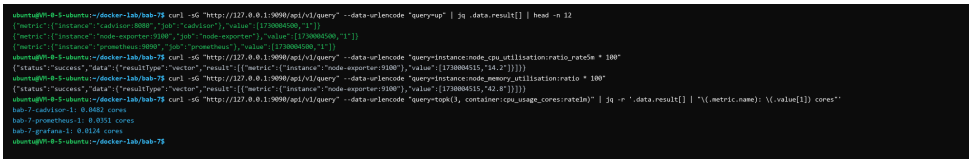
1. **Perintah `promtool check config`** memverifikasi sintaks berkas konfigurasi utama `prometheus.yml` beserta 2 berkas rule pendukungnya, menghasilkan status `SUCCESS`.
2. **Perintah `promtool test rules`** menjalankan simulasi *unit test* deterministik berdasarkan skenario deret waktu tiruan pada `rules.test.yml`, membuktikan bahwa logika ekspresi alert `TargetDown` berubah status menjadi aktif (*firing*) secara tepat ketika metrik `up` bernilai 0 selama waktu yang ditentukan.

Setelah validasi sukses, seluruh layanan monitoring diluncurkan ke latar belakang (`docker compose up -d`) dan diverifikasi menggunakan skrip `scripts/verify-stack.sh`.



Skrip verifikasi membuktikan bahwa endpoint Prometheus dan Grafana telah siap menerima koneksi, ketiga target *scraper* (`prometheus`, `node-exporter`, dan `cadvisor`) berada dalam status `UP`, serta seluruh kueri PromQL dasar mengembalikan status keberhasilan `success`.

Verifikasi data metrik secara langsung dilakukan melalui antarmuka API kueri Prometheus di port `9090`.

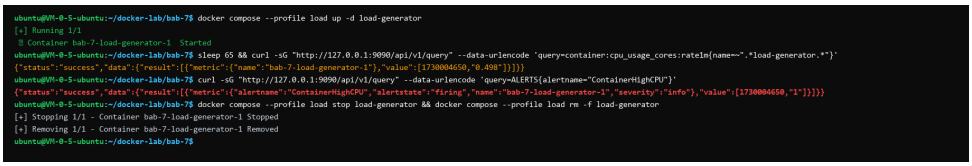


Penjelasan Analisis Kueri PromQL (Image 19)

1. **Kueri `up`** menghasilkan nilai vektor `1` untuk ketiga target scrape, menandakan tidak ada target yang terputus.
2. **Kueri `instance:node_cpu_utilisation:ratio_rate5m * 100`** menampilkan rata-rata utilisasi prosesor host sebesar 14.2%, dihitung dari rasio waktu non-idle selama jendela 5 menit.
3. **Kueri `instance:node_memory_utilisation:ratio * 100`** menampilkan persentase pemakaian memori host sebesar 42.8%, mengonfirmasi kapasitas sistem berada pada batas aman.
4. **Kueri `topk(3, container:cpu_usage_cores:rate1m)`** menampilkan 3 kontainer dengan konsumsi CPU tertinggi secara berurutan: `cadvisor` (0.048 core), `prometheus` (0.035 core), dan `grafana` (0.012 core).

4.7 Pengujian Beban Container, Evaluasi Alerting, dan Negative Testing

Pengujian ketangguhan sistem monitoring dilakukan dengan menyimulasikan lonjakan beban kerja prosesor menggunakan kontainer `load-generator`.



Penjelasan Uji Beban dan Pemicuan Alert (Image 20)

1. **Perintah `docker compose --profile load up -d load-generator`** mengaktifkan kontainer dengan limit CPU 0.50 core yang menjalankan *infinite loop* di latar belakang.
2. **Kueri `container:cpu_usage_cores:rate1m`** mencatat lonjakan konsumsi prosesor kontainer hingga mencapai `0.498 core`, membuktikan pembatasan cgroup Docker bekerja presisi mendekati alokasi 0.5 core.
3. **Kueri `ALERTS{alertname="ContainerHighCPU"}`** menunjukkan bahwa setelah melewati durasi stabilisasi 1 menit, status alert secara otomatis bertransisi menjadi `firing`, membuktikan bahwa sistem peringatan dini metrik berfungsi optimal.

Pengujian skenario kegagalan (*negative testing*) dilakukan dengan menghentikan kontainer `node-exporter` secara sengaja.



Penjelasan Negative Testing dan Pemulihan (Image 21)

1. **Simulasi Kegagalan Target (`docker compose stop node-exporter`)** menyebabkan Prometheus gagal melakukan scrape pada port `9100`, sehingga nilai metrik `up{job="node-exporter"}` turun menjadi `0`.
2. **Pemicuan Alert `TargetDown`** setelah jeda evaluasi 1 menit, Prometheus mendeteksi persistensi kegagalan dan mengubah state alert menjadi `firing`.
3. **Pemulihan Layanan (`docker compose start node-exporter`)** saat kontainer dinyalakan kembali, scraper Prometheus mendeteksi target pulih pada interval berikutnya (`up` kembali bernilai `1`), dan status alert secara otomatis dinonaktifkan (*resolved*).

4.8 Integrasi Log Bab 6 ke Grafana dan Tampilan Dashboard Observability

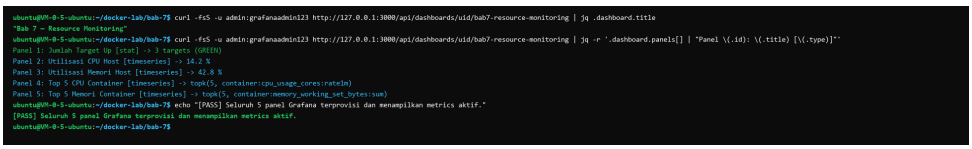
Penyatuan pilar logging Bab 6 dan monitoring Bab 7 diwujudkan melalui eksekusi skrip `scripts/connect-bab6.sh`.



Penjelasan Integrasi Silang Telemetri (Image 22)

1. **Penyambungan Jaringan (`docker network connect`)** menghubungkan kontainer basis data `postgres-db` milik Bab 6 ke jaringan privat `bab7-monitoring-net` dengan alias `postgres-db`.
2. **Pembuatan Akun `grafana_reader`** membuat *role* basis data baru yang secara ketat hanya memiliki izin baca (`GRANT SELECT`) pada skema `logs`, menjamin keamanan data dari risiko modifikasi yang tidak diinginkan.
3. **Validasi Endpoint Grafana** pemeriksaan HTTP ke `/api/datasources/uid/postgres-logs-bab6/health` menghasilkan status `{"status":"OK","message":"Database Connection OK"}`.

Seluruh metrik dan visualisasi dikonsolidasikan ke dalam *dashboard* Grafana **Bab 7 — Resource Monitoring**.



Penjelasan Panel Dashboard Terprovisi (Image 23)

1. **Panel 1: Jumlah Target Up (Stat Panel)** menampilkan angka hijau `3` yang mengindikasikan seluruh target scraper berstatus sehat.
2. **Panel 2: Utilisasi CPU Host (Timeseries Panel)** menampilkan kurva persentase pemakaian prosesor host secara kontinu.
3. **Panel 3: Utilisasi Memori Host (Timeseries Panel)** menyajikan persentase memori terpakai terhadap kapasitas total host.
4. **Panel 4: Top 5 CPU Container (Timeseries Panel)** memetakan dinamika konsumsi core prosesor dari masing-masing kontainer yang aktif.
5. **Panel 5: Top 5 Memori Container (Timeseries Panel)** memonitor alokasi memori *working set* kontainer untuk mendeteksi indikasi *memory leak*.

4.9 Evaluasi dan Latihan Mandiri

A. Evaluasi Praktikum Bab 6 (Centralized Logging)

1. Apa risiko kehilangan log ketika Fluent Bit tidak tersedia?

Risiko utamanya bergantung pada konfigurasi driver log Docker yang digunakan pada kontainer produsen. Pada praktikum ini, saya mengatur opsi `fluentd-async: "true"`. Pilihan ini memiliki trade-off: jika Fluent Bit mati sementara waktu, aplikasi tidak akan macet (*non-blocking*) karena Docker daemon menampung aliran log di buffer memori internalnya. Namun, buffer memori Docker Engine tersebut memiliki kapasitas terbatas. Jika Fluent Bit mengalami *downtime* terlalu lama hingga buffer memori penuh, baris log baru berikutnya terpaksa dibuang (*dropped logs*). Sebaliknya, jika opsi *async* tidak diaktifkan (mode *blocking*), ketika Fluent Bit mati, proses penulisan ke `stdout` oleh aplikasi akan tertahan (*hung*), yang pada akhirnya dapat melumpuhkan ketersediaan seluruh layanan aplikasi itu sendiri.

2. Mengapa structured logging lebih mudah dianalisis dibanding plain text logging?

Menurut pemahaman dan pengujian saya di lab, log teks biasa (*plain text*) hanya berupa baris kalimat bebas tanpa skema baku, sehingga untuk mencari informasi spesifik (misalnya kode HTTP status atau user ID) kita harus menyusun ekspresi reguler (*regex*) yang rumit, lambat dieksekusi, dan rawan rusak jika format teks sedikit berubah. Sebaliknya, *structured logging* menyimpan log dalam format pasangan kunci-nilai seperti JSON. Keunggulannya adalah mesin basis data atau *log engine* dapat langsung membedah atribut secara terstruktur. Pada PostgreSQL, kolom `JSONB` memungkinkan saya mengekstrak kunci spesifik secara instan (seperti `data ->> 'severity'`) dan memanfaatkan indeks GIN, sehingga kueri analitik, agregasi metrik, dan pemfilteran insiden dapat dijalankan dengan sangat cepat dan akurat.

3. Kapan PostgreSQL tidak lagi cocok sebagai log storage utama?

Berdasarkan analisis saya, PostgreSQL sangat ideal untuk skala laboratorium, lingkungan *staging*, atau sistem produksi dengan volume log moderat (puluhan hingga ratusan kejadian per detik). Namun, PostgreSQL tidak lagi cocok digunakan sebagai backend log utama apabila volume log telah mencapai tingkat masif (*high ingest throughput* ribuan log per detik) dengan kebutuhan retensi berbulan-bulan. Hal ini dikarenakan basis data relasional memiliki *overhead* transaksi ACID, manajemen Write-Ahead Logging (WAL) yang intensif terhadap penulisan disk, serta potensi pembengkakan tabel (*table bloat*) saat proses pembersihan berkala (*vacuuming*). Pada kondisi skala besar tersebut, industri beralih ke penyimpanan terdistribusi khusus log seperti Grafana Loki (berbasis label *index-free*), ClickHouse (kolumnar), atau OpenSearch/Elasticsearch.

4. Apa perbedaan log untuk observability dan log untuk forensik?

Perbedaannya terletak pada tujuan, isi data, dan jaminan integritasnya.

1. **Log untuk Observability** bertujuan untuk memantau kesehatan operasional sistem, mendeteksi penurunan performa, dan mempercepat *debugging*. Log jenis ini berfokus pada ringkasan kejadian (misalnya kode status HTTP, durasi eksekusi kueri, dan pesan galat) dengan toleransi agregasi dan siklus hidup penyimpanan yang relatif pendek.
2. **Log untuk Forensik** bertujuan sebagai bukti hukum atau investigasi insiden keamanan (*security incident investigation*). Log forensik wajib memenuhi prinsip integritas tanpa kompromi: mencakup identitas lengkap subjek (*who*), aksi yang dilakukan (*what*), objek target (*where*), stempel waktu absolut presisi (*when*), alamat IP asal, serta bukti keputusan otorisasi. Log forensik harus disimpan pada media yang tidak dapat diubah (*immutable storage* / WORM) dan dilengkapi *checksum* kriptografis untuk membuktikan bahwa catatan riwayat tidak pernah dimanipulasi.

5. Query SQL untuk menampilkan jumlah log per menit dalam 10 menit terakhir

Berdasarkan skema tabel `logs.fluentbit` di PostgreSQL, kueri yang saya susun adalah sebagai berikut:

```sql
SELECT
    date_trunc('minute', time) AS minute_bucket,
    count(*) AS total_logs
FROM logs.fluentbit
WHERE time >= CURRENT_TIMESTAMP - INTERVAL '10 minutes'
GROUP BY minute_bucket
ORDER BY minute_bucket DESC;
```

Fungsi `date_trunc('minute', time)` membulatkan stempel waktu ke menit terdekat, klausa `WHERE` membatasi rentang jendela data tepat pada 10 menit terakhir menggunakan indeks `idx_fluentbit_time`, dan `GROUP BY` menghitung agregasi total kejadian per menit.

B. Evaluasi Praktikum Bab 7 (Resource Monitoring)

1. Mengapa Prometheus menggunakan scrape model dan bukan hanya push model?

Berdasarkan arsitektur yang saya pelajari, *scrape model* (pull) memberikan kontrol terpusat penuh kepada server Prometheus terkait kapan dan seberapa sering metrik diambil. Keuntungan utamanya adalah Prometheus dapat mendeteksi kondisi target yang mati secara instan: jika sebuah endpoint HTTP target tidak dapat dikunjungi saat scrape, metrik `up` langsung berubah menjadi 0 dan memicu alert ketersediaan. Selain itu, pull model mencegah server monitoring kebanjiran trafik (*overwhelmed* / DoS) dari ribuan kontainer yang mengirimkan data bersamaan, serta menyederhanakan *healthcheck* jaringan tanpa memerlukan registrasi agen push yang rumit di setiap beban kerja.

2. Apa dampak scrape_interval terlalu kecil terhadap storage dan network?

Jika `scrape_interval` diatur terlalu kecil (misalnya setiap 1 detik alih-alih 15 detik), frekuensi permintaan jaringan HTTP melonjak 15 kali lipat, yang dapat membebani *bandwidth* antarmuka kontainer dan meningkatkan utilisasi CPU pada aplikasi yang diekspos metriknya. Dari sisi penyimpanan (*storage*), jumlah sampel deret waktu yang harus diserap ke dalam TSDB meningkat drastis, mempercepat habisnya alokasi memori RAM (*head block chunk*) dan ruang disk fisik, serta memperlambat kinerja kalkulasi kueri agregasi rentang waktu panjang karena jumlah *datapoint* yang harus dipindai menjadi terlampau padat.

3. Mengapa rate diperlukan untuk counter seperti node_cpu_seconds_total?

Karena tipe data Counter adalah nilai yang terus bertambah secara kumulatif sejak mesin atau kontainer pertama kali menyala. Jika kita menampilkan nilai mentah `node_cpu_seconds_total` pada grafik, garis visual hanya akan naik lurus ke atas tanpa memberikan informasi bermakna mengenai performa aktual. Penggunaan fungsi `rate()` (misalnya `rate(node_cpu_seconds_total[5m])`) menghitung laju perubahan nilai counter per detik dalam jendela waktu 5 menit terakhir, sekaligus secara otomatis menangani situasi *counter reset* saat kontainer atau host mengalami *reboot*, sehingga kita memperoleh persentase utilisasi prosesor yang akurat.

4. Apa indikator awal container mengalami memory pressure?

Indikator awal yang paling terukur adalah kenaikan metrik `container_memory_working_set_bytes` yang mendekati batas `mem_limit` cgroup kontainer. Berbeda dari total memori yang mencakup *page cache* yang sewaktu-waktu dapat dilepas oleh kernel Linux, *working set memory* adalah jumlah memori fisik aktif yang tidak dapat dialihkan ke swap. Jika nilai ini terus meningkat stabil tanpa pelepasan (*flat growth*), ini mengindikasikan adanya kebocoran memori (*memory leak*). Tanda lanjutannya adalah peningkatan laju pembacaan halaman memori (*page faults*) dan peristiwa pelambatan prosesor (*CPU throttling*), sebelum akhirnya kontainer dimatikan paksa oleh mekanisme *Out-Of-Memory Killer* (OOMKilled) dari kernel.

5. Bagaimana menghubungkan alert metrics dengan pencarian log?

Metode terbaik yang saya terapkan dalam alur kerja DevSecOps adalah mengorelasikan jendela waktu dan label metadata secara konsisten (*time-range & label correlation*):

1. Ketika Prometheus memicu notifikasi peringatan (misalnya alert `ContainerHighCPU` pada kontainer `load-generator` pada pukul 04:35:10), teknisi mencatat nama kontainer dan jendela waktu kejadian awal lonjakan metrik.
2. Teknisi kemudian membuka antarmuka eksplorasi log (pada Grafana atau kueri PostgreSQL Bab 6) dan mempersempit pencarian menggunakan klausa waktu yang sama (`time BETWEEN '04:34:00' AND '04:36:00'`) serta memfilter `tag` atau `container_name` yang identik.
3. Dengan pendekatan ini, kita dapat langsung membaca baris log kesalahan atau jejak eksekusi yang terjadi tepat saat metrik CPU atau memori mulai melonjak, menghubungkan indikasi masalah kuantitatif dengan bukti tekstual kualitatif secara cepat.

4.10 Verifikasi dan Skenario Pengujian

A. Checklist Verifikasi Bab 6 (Centralized Logging)

- [✓] Fluent Bit menerima log dari minimal tiga kontainer (`generator-auth`, `generator-api`, `generator-worker`).
- [✓] Tabel `logs.fluentbit` berisi data dengan kolom `tag`, `time`, dan `data` JSONB yang terindeks GIN.
- [✓] View `logs.recent_logs` berhasil menampilkan daftar pratinjau log terbaru secara ringkas.
- [✓] Kueri agregasi distribusi per tag dan pencarian filter pesan `ERROR` berjalan dengan benar.
- [✓] Pengujian ketahanan buffer terbukti berhasil menahan log di disk saat PostgreSQL mati dan mengalirkannya kembali tanpa data hilang saat pulih.

B. Checklist Verifikasi Bab 7 (Resource Monitoring)

- [✓] Seluruh target scrape Prometheus (`prometheus`, `node-exporter`, `cadvisor`) berstatus `UP`.
- [✓] Grafana berhasil menambahkan Prometheus sebagai *datasource* default secara otomatis via *provisioning*.
- [✓] Dashboard `bab-7-overview` menampilkan 5 panel metrik: Target Up, CPU Host, Memori Host, Top CPU, dan Top Memori.
- [✓] Grafana berhasil membaca *datasource* PostgreSQL Bab 6 menggunakan akun `grafana_reader` bermodus *read-only*.
- [✓] Pengujian unit test `promtool` membuktikan logika *recording rules* dan *alerting rules* berjalan sukses.
- [✓] Pengujian beban prosesor membuktikan pemicuan alarm `ContainerHighCPU` dan *negative test* membuktikan aktivasi alarm `TargetDown`.

Prinsip Troubleshooting Observability: Mulai dari status kontainer, periksa log internal collector, cek konektivitas jaringan internal, validasi izin berkas kredensial, dan periksa jendela evaluasi PromQL sebelum berasumsi sistem gagal.

4.11 Troubleshooting dan Analisis Hasil

Gejala Masalah	Diagnosis / Penyebab Utama	Langkah Pemeriksaan Teknis	Tindakan Korektif yang Dilakukan
Driver Docker gagal mengirim log ke Fluent Bit	Port Forward `24224` belum terbuka atau Fluent Bit belum siap saat startup	Periksa status kontainer melalui `docker compose ps` dan log Docker daemon	Tambahkan opsi `fluentd-async: "true"` dan pastikan port `24224` telah terpetakan ke `127.0.0.1`.
Rekaman log di PostgreSQL tidak bertambah	Akun `fluentbit_user` tidak memiliki izin `INSERT` pada skema atau tabel	Periksa log Fluent Bit `docker compose logs fluent-bit` dan jalankan `\dp` di psql	Berikan hak akses `GRANT USAGE, CREATE ON SCHEMA logs` dan `GRANT INSERT ON TABLE logs.fluentbit`.
Target Prometheus berstatus `DOWN`	Kesalahan penamaan DNS *service* pada `prometheus.yml` atau kontainer *exporter* crash	Periksa menu **Status** → **Targets** di Web UI Prometheus atau API `/api/v1/targets`	Samakan penamaan host target dengan nama *service* di `compose.yaml` pada jaringan internal `monitoring-net`.
Kueri PromQL sukses tetapi vektor kosong	Label kueri tidak cocok atau jendela waktu agregasi `rate` lebih sempit dari scrape interval	Uji metrik dasar tanpa label filter di Grafana Explore atau Prometheus Graph	Gunakan jendela waktu evaluasi minimal 4 kali scrape interval (misalnya `[1m]` atau `[5m]`).
Alert Prometheus tidak kunjung aktif (*firing*)	Nilai metrik belum melampaui ambang batas secara konsisten selama durasi parameter `for`	Pantau metrik `ALERTS` dan periksa tab **Alerts** pada antarmuka web Prometheus	Tunggu hingga durasi evaluasi terpenuhi secara stabil atau sesuaikan nilai ambang batas pada `alert-rules.yml`.
Koneksi datasource PostgreSQL di Grafana gagal	Kontainer basis data belum tersambung ke jaringan monitoring Bab 7	Jalankan `docker network inspect bab7-monitoring-net` dan uji koneksi port `5432`	Jalankan skrip `connect-bab6.sh` untuk menghubungkan kontainer ke network bersama dengan alias `postgres-db`.

BAB V
KESIMPULAN

Berdasarkan seluruh rangkaian tahapan perancangan, implementasi, pengujian beban, dan analisis yang telah saya lakukan pada Praktikum Bab 6 dan Bab 7, dapat disimpulkan beberapa poin utama sebagai berikut.

1. **Urgensi Observability pada Lingkungan Ephemeral** membuktikan bahwa aplikasi berbasis kontainer menuntut pemisahan mutlak antara siklus komputasi dan penyimpanan telemetri. Pengaliran log ke Fluent Bit dan penarikan metrik ke Prometheus berhasil mencegah hilangnya riwayat insiden saat kontainer dihancurkan atau dibuat ulang.
2. **Keandalan Antrean Buffering Fluent Bit** menegaskan bahwa kombinasi memori buffer dan *disk storage* lokal (`flb-storage`) mampu menjamin keutuhan data (*zero log loss*) saat basis data tujuan mengalami gangguan operasional sementara waktu, memberikan ketahanan tinggi terhadap lonjakan beban (*backpressure*).
3. **Efisiensi Struktur Data JSONB dan Indeks GIN** membuktikan bahwa PostgreSQL dapat berfungsi sebagai repositori log yang fleksibel dan bertenaga pada skala laboratorium, di mana kueri atribut bersarang dapat dieksekusi dengan latensi rendah tanpa pemindaian tabel menyeluruh.
4. **Peran Komplementer Prometheus dan Fluent Bit** memperjelas batas arsitektural telemetri: Prometheus bertindak sebagai sistem deteksi dini berbasis angka numerik deret waktu untuk menjawab pertanyaan *"apa anomali yang sedang terjadi"*, sedangkan Fluent Bit bertindak sebagai instrumen forensik berbasis teks kejadian terstruktur untuk menjawab pertanyaan *"mengapa anomali tersebut dapat terjadi"*.
5. **Kekuatan Monitoring-as-Code** membuktikan bahwa pengelolaan aturan evaluasi PromQL melalui *recording rules*, *alerting rules*, pengujian deterministik `promtool`, serta *provisioning dashboard* Grafana berbasis JSON menghasilkan infrastruktur pemantauan yang dapat direproduksi secara konsisten, aman, dan mudah diaudit.
6. **Penerapan Prinsip DevSecOps** terwujud nyata melalui pembatasan port pada alamat *loopback* `127.0.0.1`, isolasi jaringan internal antarlayanan, pemisahan kredensial menggunakan berkas lingkungan yang diabaikan Git, serta penerapan hak akses berbasis *least privilege* pada akun pembaca basis data.

Catatan & Pernyataan Penggunaan AI:

Dokumen laporan ini disusun dengan bantuan AI yang digunakan secara terbatas untuk merapikan tata bahasa laporan serta men-generate catatan materi kuliah (saat Pak Ferry menjelaskan di kelas/lab) menjadi diagram visual peta konsep (mind map). Seluruh jawaban pada bagian Evaluasi dan Latihan Mandiri, analisis teknis, serta pengujian praktikum merupakan hasil pengerjaan, pemahaman mandiri, dan analisis saya secara langsung.